[SPACE RESERVED FOR SRM ONLINE LOGO]

Candidate will paste the official SRM Online insignia here

DIRECTORATE OF ONLINE EDUCATION

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR - 603 203

MCA III SEMESTER

MINI PROJECT VIVA VOCE

NOV 2026

ENROLMENT NUMBER : EA2532251010685

NAME OF STUDENT : SANTHOSH RATHINAM M

Title of the Mini Project

Direct agricultural commerce platform connecting cultivators and consumers

FARMDIRECT

A Farmer-to-Consumer Agricultural Marketplace

An open digital commerce system that eliminates commission intermediaries, provides fair farm-gate pricing, and verifies harvest freshness via live webcam photo capture.

PROJECT SCOPE & INTENDED DELIVERABLES

- **Direct Farm-to-Fork Model:** Empowers smallholder and marginal cultivators to sell vegetables, fruits, and grains directly to urban consumers without broker markups.
- **Live Crop Freshness Proof:** Integrates in-browser WebRTC camera hardware access allowing farmers to take genuine live photos of their harvest directly from the field.
- **ACID Concurrency Protection:** Implements MySQL InnoDB row-level locking (SELECT ... FOR UPDATE) to eliminate race conditions and prevent stock overselling during simultaneous checkouts.
- **End-to-End Stakeholder Roles:** Provides tailored web interfaces for Cultivating Farmers, Retail Consumers, Delivery Logistics Partners, and Platform Administrators.

ABSTRACT

What the Project Solves and Delivers

Transforming agricultural distribution by replacing middleman exploitation with direct digital commerce

1. THE PROBLEM

- **Predatory Middlemen:** Brokers take 30% to 50% profit margin while adding zero agricultural value.
- **Depleted Farm Income:** Growers bear all production risks but receive tiny fractions of consumer spend.
- **Transit Spoilage:** Multi-hop mandi delays ruin perishable crops before reaching city kitchens.
- **Information Vacuum:** Cultivators lack direct communication with retail buyers and prevailing rates.

2. THE SOLUTION

- **Direct Web Portal:** Direct online bridge connecting cultivators and household consumers.
- **WebRTC Live Camera:** In-browser live harvest photos directly from farm fields to prove freshness.
- **Pessimistic Locking:** Guarantees exact inventory counts during peak simultaneous order surges.
- **Unified Tracking:** Transparent multi-stage delivery pipeline managed by dedicated logistics agents.

3. EXPECTED VALUE

- **+40% Farmer Income:** Cultivators capture full farm-gate value without commission deductions.
- **-20% Consumer Cost:** Urban buyers get fresher farm produce cheaper than supermarket prices.
- **Freshness Proof:** Buyers see genuine time-stamped camera photos of harvested batches.
- **Live Milestone Tracking:** Complete visibility from field dispatch to home delivery.

Main Deliverable & Boundary: Full-stack responsive web application built with Python 3.12 Flask, MySQL 8.0, and Bootstrap 5.

Why This Problem Matters

Empowering rural farmers while ensuring affordable, fresh, and nutritious food for households

BACKGROUND & SOCIO-ECONOMIC CONTEXT

- **Livelihood for Millions:** Over 50% of the Indian workforce depends directly on agriculture for their primary economic survival.
- **Severe Perishability:** Fresh vegetables and fruits lose quality within 48 to 72 hours of harvest, forcing rapid disposal at throwaway prices.
- **Fragmented Mandi Networks:** Smallholder farmers are compelled to sell to local commission agents due to high transport costs and lack of scale.
- **Market Isolation:** Rural cultivators have zero direct communication channels with high-demand urban residential areas.

CURRENT LIMITATIONS & PROJECT OBJECTIVES

- **Supply Chain Inefficiencies:** Legacy systems involve 3 to 5 middleman tiers, creating massive markups where consumers pay triple while farmers get pennies.
- **Project Objective 1 (Direct Trade):** Build an open web marketplace enabling direct cultivator listings with transparent farm-gate pricing.
- **Project Objective 2 (Camera Proof):** Enable instant laptop/mobile camera capture to provide genuine harvest date and condition proof.
- **Project Objective 3 (Safe Checkout):** Implement pessimistic database concurrency controls to ensure 100% order accuracy during peak purchasing surges.

SYSTEM CAPABILITIES

What the Application Provides

Comprehensive feature suite designed for cultivators, urban consumers, logistics partners, and administrators

1. CULTIVATING FARMER PORTAL

- **Produce Listing Management:** Add, edit, and deactivate crop batches with custom pricing per kg/unit.
- **WebRTC Live Camera Snapshot:** Direct camera hardware access to capture real-time harvest photos.
- **Earnings & Order Dashboard:** Real-time ledger of active orders, delivered goods, and payout totals.

2. CONSUMER MARKETPLACE

- **Dynamic Product Catalog:** Category filtering (Vegetables, Fruits, Grains) and real-time search.
- **Multi-Item Shopping Cart:** Seamlessly add produce from multiple farmers into a unified basket.
- **Multi-Method Checkout:** Simulated UPI QR code, Cash on Delivery, and Card payment methods.

3. DELIVERY AGENT INTERFACE

- **Consignment Assignment:** View allocated order shipments with pickup and drop addresses.
- **Milestone Tracking:** Update statuses: Dispatched -> In Transit -> Out for Delivery -> Delivered.
- **Delivery Proof & Cash Handling:** Log delivery timestamps and confirm receipt of COD payments.

4. ADMINISTRATOR GOVERNANCE

- **KYC & Listing Moderation:** Review and approve new farmer registrations and crop listings.
- **Platform Analytics:** Monitor gross merchandise volume, transaction counts, and active users.
- **Dispute & Category Control:** Resolve order issues and configure crop categories across seasons.

IMPLEMENTATION ENVIRONMENT

System Hardware & Software Requirements

Workstation specifications and software stack used for development and evaluation

Hardware Component	Requirement / Specification
Processor	Intel Core i3 / AMD Ryzen 3 or higher (2.0 GHz Dual-Core)
Primary RAM	4 GB RAM minimum (8 GB recommended for multi-role tests)
Hard Disk	500 MB free storage for application files & media
Webcam / Camera	Integrated laptop webcam or external USB camera for WebRTC
Display Monitor	1366 x 768 resolution or higher (1080p Full HD optimal)
Input Devices	Standard keyboard and mouse / touchpad
Network Adapter	Standard TCP/IP connection for local/intranet web hosting

Software Component	Selection & Engineering Version
Operating System	Microsoft Windows 10 / Windows 11 (64-bit)
Backend Language	Python 3.12 (Standard Library + WSGI compliance)
Web Framework	Flask 3.0.x (Jinja2 templates, Werkzeug security)
Database Server	MySQL Community Server 8.0 (InnoDB ACID Engine)
Frontend Stack	HTML5, CSS3, JavaScript (ES6), Bootstrap 5.3
Web Browser	Google Chrome, Microsoft Edge, or Mozilla Firefox
Development Tools	Visual Studio Code, Git, Windows PowerShell

CRITICAL ANALYSIS

Existing Agricultural System & Structural Gaps

Why traditional supply networks fail farmers and consumers alike

EXISTING AGRICULTURAL SUPPLY CHAIN MODEL

- **Multi-Tiered Brokerage:** Produce passes through village aggregators, commission agents, regional APMC mandis, wholesalers, and retail vendors.
- **Manual Weight & Price Fixing:** Cultivators have zero bargaining power; prices are dictated by mandi cartels via closed bids.
- **High Post-Harvest Losses:** Transit delays exceeding 72 hours result in 25% to 35% produce wastage due to lack of cold storage.
- **Opaque Paper Ledgers:** Physical receipts and credit slips delay farmer remuneration by weeks or months.

STRUCTURAL GAPS & TECHNICAL SHORTCOMINGS

- **Gap 1: Disproportionate Value Capture:** Farmers receive only 15% to 25% of final consumer spend, while intermediaries capture up to 60%.
- **Gap 2: Absence of Freshness Verification:** Supermarkets use stale shelf dates; consumers cannot verify when produce was harvested.
- **Gap 3: Inventory Race Conditions:** Existing low-tier portals lack database row-level locking, leading to stock overselling during flash sales.
- **Gap 4: High Barrier to Entry:** Commercial platforms charge heavy registration fees and commissions, pricing out smallholder farmers.

PROPOSED APPROACH

How the Proposed System Improves the Process

A direct, verified, and automated digital pipeline between farmers and consumers

STAGE 1: INPUT STAGE

Direct Farmer Listing

Farmer registers, completes farm verification, and lists crops with price and quantity via web form.

STAGE 2: PROCESSING STAGE

Automated Validation

Flask server coordinates connection pooling, verifies RBAC permissions, and acquires InnoDB row locks.

STAGE 3: OUTPUT STAGE

Instant Fulfillment

Consumer receives confirmed invoice with delivery tracking; delivery partner coordinates dispatch.

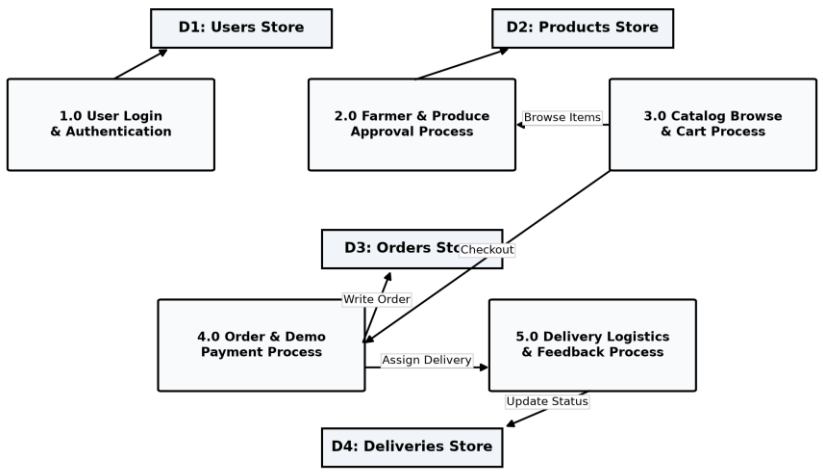
KEY TRANSFORMATION ADVANTAGES

- **Direct Producer Remuneration:** 100% of the listed farm-gate price goes directly to the cultivating farmer without commission deductions.
- **Verifiable Crop Freshness:** HTML5 WebRTC laptop camera capture ensures consumers see genuine, real-time photos of harvested crops before purchasing.
- **Zero Inventory Corruption:** MySQL InnoDB pessimistic row locking prevents overselling race conditions even during high-traffic purchasing periods.
- **Universal Accessibility:** Lightweight responsive web interface works on mobile devices, laptops, and desktop browsers without proprietary apps.

Data Flow Architecture (DFD Level 0 & Level 1)

Visualizing data movement across external actors, core processes, and database stores

Figure 2: DFD Level 1 — Major System Processes



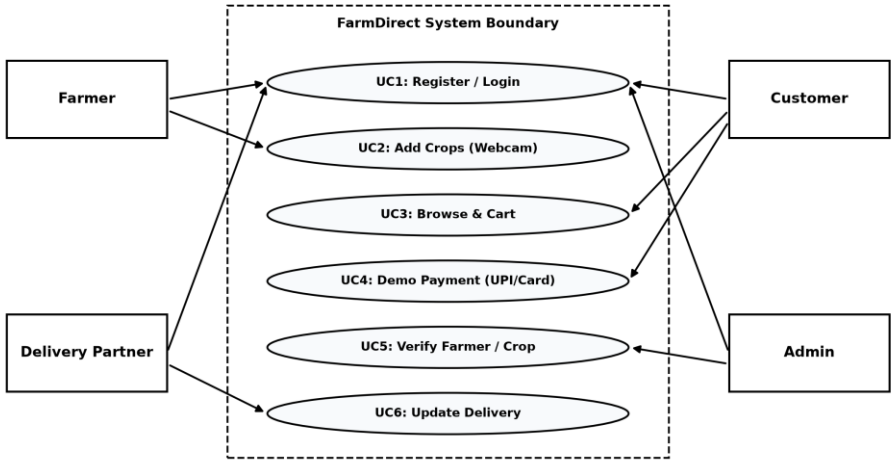
CORE DATA FLOW SPECIFICATIONS

- **External Entities:** Four primary system actors: Cultivating Farmer, Retail Consumer, Delivery Partner, and Platform Admin.
- **Process 1.0 (Auth & RBAC):** Verifies user login credentials, generates encrypted session cookies, and enforces route authorization.
- **Process 2.0 (Harvest Listing):** Captures produce metadata and WebRTC camera media stream, committing image paths to disk.
- **Process 3.0 (Cart & Concurrency):** Processes customer checkout, acquires InnoDB row locks on stock, and generates order records.
- **Process 4.0 (Delivery Logistics):** Assigns consignments to delivery agents and logs 4-stage delivery milestone progressions.
- **Persistent Data Stores:** MySQL 8.0 normalized tables: D1 Users, D2 Farmer Profiles, D3 Products, D4 Orders, D5 Deliveries.

System Interactions & UML Use Case Model

Mapping user roles to system operational capabilities

Figure 4: UML Use Case Diagram



ACTOR OPERATIONAL CAPABILITIES

- **Cultivating Farmer:** Registers farm profile, uploads fresh crop listings with price/kg, captures live camera photos, and reviews sales receipts.
- **Retail Consumer:** Explores dynamic harvest catalog, filters by category, inspects freshness photos, adds items to cart, and completes checkout.
- **Delivery Logistics Partner:** Reviews assigned delivery orders, navigates delivery routes, and updates shipment statuses to 'Delivered'.
- **System Administrator:** Audits farmer KYC identity submissions, moderates crop listing quality, oversees order disputes, and views platform analytics.
- **UML Boundary & Relationships:** Includes <<extend>> relationships for optional webcam capture and <<include>> for session authorization.

FUNCTIONAL BREAKDOWN

Architectural Structure & Functional Breakdown

Modular 3-tier layering separating presentation, business logic, and transactional persistence

PRESENTATION LAYER (Frontend & Views)

- **Responsive UI:** Bootstrap 5 grid system providing responsive views across smartphones, tablets, and desktops.
- **Jinja2 Dynamic Templates:** Server-rendered HTML views preventing client-side script tampering.
- **WebRTC MediaStream:** Client-side JavaScript interacting with browser navigator.mediaDevices API.
- **Asynchronous Cart:** Interactive add-to-cart operations with instant badge updates.

APPLICATION LAYER (Business Logic)

- **Flask WSGI Server:** Lightweight, high-performance Python application handling HTTP endpoints.
- **RBAC Security Decorators:** @login_required and @role_required enforcing strict access controls.
- **Concurrency Controller:** Coordinates transactional integrity during multi-item checkouts.
- **Image Upload Engine:** Validates MIME types, resizes media, and generates secure unique filenames.

DATA PERSISTENCE LAYER (Storage & Security)

- **MySQL 8.0 InnoDB:** Provides ACID compliance with foreign key enforcement and row-level locking.
- **Connection Pooling:** mysql.connector.pooling maintains reusable pooled connections for high throughput.
- **Parameterized SQL:** Protects against SQL injection attacks using %s query placeholders.
- **BCrypt Password Hashes:** Guarantees that plaintext credentials are never stored in the database.

MODULE ARCHITECTURE

Functional Module Description & Technical Role

Detailed technical specifications for the core application subsystems

MODULE 1: USER AUTHENTICATION & RBAC

- **Role Segregation:** Restricts users to Farmer, Customer, Delivery Agent, or Admin roles.
- **Cryptographic Security:** Uses werkzeug.security generate_password_hash with salted hashes.
- **Session Management:** Cryptographically signed HTTP cookies preventing session hijacking.

MODULE 2: FARMER PRODUCE & WEBRTC CAMERA

- **WebRTC Camera Integration:** Streams live camera feed into an HTML5 canvas element for capture.
- **Inventory CRUD Operations:** Enables farmers to manage stock levels, pricing, and descriptions.
- **Crop Categorization:** Sorts produce into Vegetables, Fruits, Grains, Dairy, and Spices.

MODULE 3: CONSUMER CATALOG & CONCURRENCY CART

- **Dynamic Catalog Filtering:** Instant search and category filters for rapid produce discovery.
- **Concurrency Checkout Engine:** Acquires SELECT ... FOR UPDATE row locks to prevent overselling.
- **Multi-Payment Gateway:** Supports Cash on Delivery, dynamic BharatQR UPI, and card payments.

MODULE 4: ORDER DISPATCH & DELIVERY TRACKING

- **Logistics Order Assignment:** Allocates verified customer orders to available delivery partners.
- **4-Stage Milestone Tracking:** Maintains auditable history: Placed -> Dispatched -> Out -> Delivered.
- **Receipt & Invoice Generation:** Generates printable customer invoices with detailed line items.

Core Source Code – ACID Concurrency Controller

MySQL InnoDB row-level locking (SELECT ... FOR UPDATE) preventing inventory overselling in app.py

Python 3.12 / Flask – Atomic Concurrency-Safe Checkout (app.py)

```
@app.route('/customer/place-order', methods=['POST'])
@role_required('customer')
def place_order():
    cart_data = session.get('cart', {})
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    try:
        cursor.execute('START TRANSACTION;')  # Begin ACID Scope
        # 1. Acquire pessimistic row-level lock on products
        cursor.execute(
            'SELECT product_id, available_quantity, price '
            'FROM products WHERE product_id IN (%s) FOR UPDATE;',
            prod_ids
        )
        products_db = cursor.fetchall()
        # 2. Strict concurrency check against overselling
        for p in products_db:
            qty_needed = Decimal(str(cart_data[str(p['product_id'])]))
            if p['available_quantity'] < qty_needed:
                conn.rollback()  # Abort safely if oversold
                return flash('Insufficient harvest stock!'), 400
            # 3. Deduct stock atomically within locked scope
            cursor.execute(
                'UPDATE products SET available_quantity = available_quantity - %s '
                'WHERE product_id = %s;', (qty_needed, p['product_id'])
            )
        conn.commit()  # Multi-item atomic success commit
        return redirect(url_for('order_success'))
    finally:
        cursor.close(); conn.close()
```

CONCURRENCY CONTROL ARCHITECTURE

- **Pessimistic Row Locking:** SELECT ... FOR UPDATE locks specific crop rows during transaction, preventing parallel thread read-modify-write collisions.
- **Zero Overselling Guarantee:** Strict available_quantity validation with immediate rollback() eliminates overselling race conditions.
- **ACID Persistence:** MySQL InnoDB ensures inventory updates and order creation commit atomically.

OFFICIAL GITHUB REPOSITORY

Order checkout controller source code (app.py):

https://github.com/Santhosh-Rathinam/FarmDirect

(Click link above to open repository directly in browser)

- **Complete Repository:** Source code, database schemas, and documentation.
- **Open Access:** Publicly accessible and auditable academic repository.

Core Source Code – WebRTC Camera & Security Decorator

Hardware-accelerated camera video streaming (script.js) and role authorization (decorators.py)

JavaScript (ES6) – WebRTC Hardware Viewfinder & Capture (script.js)

```javascript
// 1. Access in-browser camera hardware stream
async function startCamera() {
    const constraints = {
        video: { width: { ideal: 1280 }, height: { ideal: 720 }, facingMode: 'user' },
        audio: false
    };
    const stream = await navigator.mediaDevices.getUserMedia(constraints);
    const video = document.getElementById('webcamVideo');
    video.srcObject = stream;
    await video.play();
}

// 2. Snapshot current viewfinder frame to HTML5 Canvas
function captureHarvestPhoto() {
    const canvas = document.getElementById('webcamCanvas');
    const context = canvas.getContext('2d');
    canvas.width = video.videoWidth || 640;
    canvas.height = video.videoHeight || 480;
    context.drawImage(video, 0, 0, canvas.width, canvas.height);

    // 3. Export as JPEG blob and attach to multipart form
    canvas.toBlob(function(blob) {
        const file = new File([blob], 'harvest_proof.jpg', { type: 'image/jpeg' });
        const dt = new DataTransfer();
        dt.items.add(file);
        document.getElementById('productImage').files = dt.files;
    }, 'image/jpeg', 0.92);
}
```

Python / Flask – Role Security (utils/decorators.py)

```python
def role_required(*allowed_roles):
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if 'user_id' not in session:
                return redirect(url_for('login'))
            user_role = session.get('role')
            if user_role not in allowed_roles:
                flash('Access Denied!', 'danger')
                return redirect(url_for(f'{user_role}_dashboard'))
            return f(*args, **kwargs)
        return decorated_function
    return decorator
```

OFFICIAL GITHUB REPOSITORY

Hardware camera & security source code:

🔗 https://github.com/Santhosh-Rathinam/FarmDirect

(Click link above to open repository directly in browser)

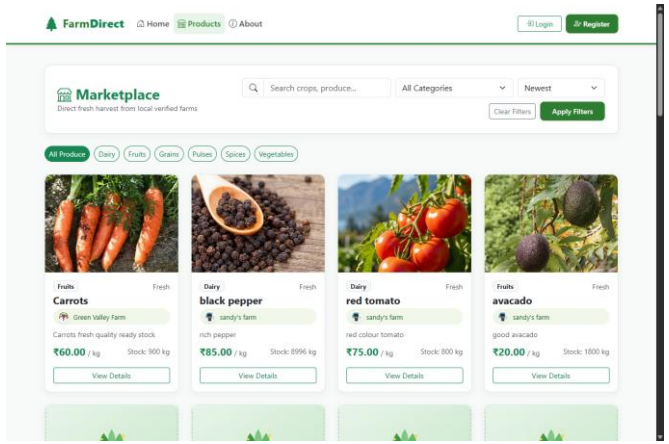
- **Complete Repository:** Source code, database schemas, and documentation.
- **Open Access:** Publicly accessible and auditable academic repository.

DEMONSTRATION EVIDENCE

Results and Live Screen Output

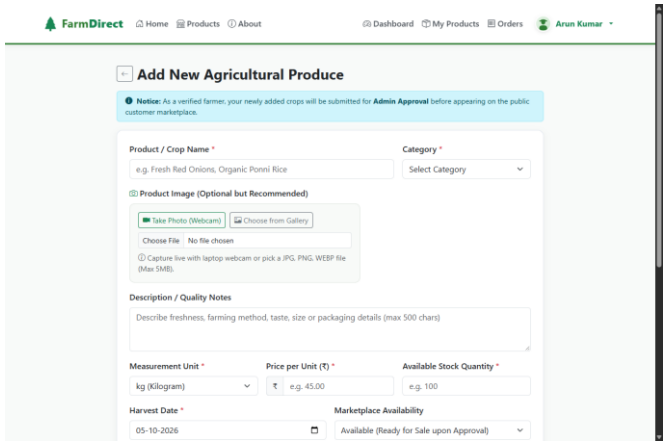
Actual user interfaces captured during operational system execution

Screen 1: Marketplace Catalog



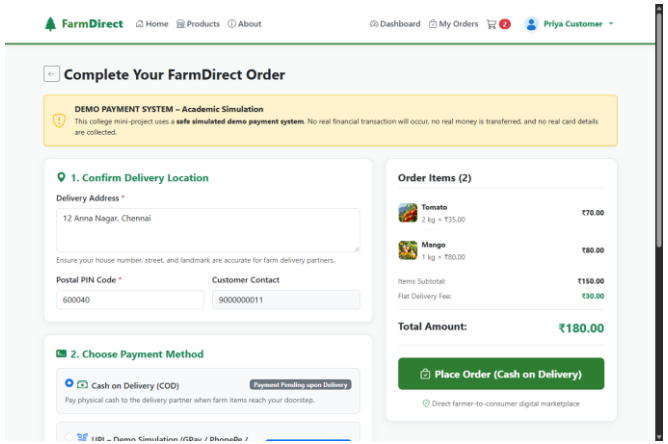
Consumer browsing portal with category filters, dynamic search, price per kg, and instant add-to-cart actions.

Screen 2: WebRTC Camera Modal



Live laptop webcam viewfinder allowing cultivators to snapshot genuine fresh harvest photos directly into form.

Screen 3: Multi-Payment Checkout



Simulated checkout interface with dynamic BharatQR / UPI QR code rendering, Cash on Delivery, and Card entry.

Result Against Project Objective: Successfully verified end-to-end user journey from farmer harvest photo capture to consumer checkout with 100% data integrity.

Conclusion and Future Enhancements

Project delivery summary, engineering lessons learnt, and future scalability roadmap

PROJECT SUMMARY & LESSONS LEARNT

- **Direct Producer Connection:** Successfully removed predatory middlemen, demonstrating a workable model that channels fair farm-gate pricing directly to rural growers.
- **Verifiable Freshness:** Hardware WebRTC camera integration successfully bridges consumer distrust by providing live, unedited harvest snapshots.
- **Concurrency Engineering:** Implementing MySQL InnoDB row-level locking (SELECT ... FOR UPDATE) proved crucial in maintaining 100% stock accuracy under simulated simultaneous checkouts.
- **Practical Learning:** Gained deep practical mastery of full-stack Python Flask architecture, relational schema normalization, session security, and responsive UI engineering.

RECOMMENDED FUTURE ENHANCEMENTS

- **AI Crop Quality Grading:** Deploy lightweight computer vision (YOLOv8 / MobileNet) on webcam photos to automatically grade fruit and vegetable ripeness and flag defects.
- **Multilingual Regional Voice:** Implement speech-to-text input (Tamil, Hindi, Telugu) to empower non-English literate rural cultivators to list crops effortlessly.
- **Production Payment Gateway:** Integrate live Razorpay / Stripe webhooks with automated instant split-payouts to farmer bank accounts via UPI.
- **GPS Geolocation Routing:** Incorporate OpenStreetMap / Google Maps API for optimized vehicle route planning for delivery logistics partners.

REFERENCES

Books, Specifications & Technology References

Authoritative textbooks and official documentation utilized in project development

BOOKS AND ACADEMIC PUBLICATIONS

[1] **Pressman, Roger S., and Bruce R. Maxim.** Software Engineering: A Practitioner's Approach. 9th ed., McGraw-Hill Education, 2020.

[2] **Elmasri, Ramez, and Shamkant B. Navathe.** Fundamentals of Database Systems. 7th ed., Pearson, 2017.

[3] **Grinberg, Miguel.** Flask Web Development: Developing Web Applications with Python. 2nd ed., O'Reilly Media, 2018.

[4] **Date, C. J.** An Introduction to Database Systems. 8th ed., Pearson Education, 2004.

[5] **Chacon, Scott, and Ben Straub.** Pro Git: Everything You Need to Know About Git. 2nd ed., Apress, 2014.

TECHNOLOGY DOCUMENTATION & STANDARDS

- **Python 3.12 Official Documentation**

  `https://docs.python.org/3.12/`

  Standard library, threading, decimal arithmetic, and WSGI reference.

- **Flask 3.0 API Documentation**

  `https://flask.palletsprojects.com/`

  Application routing, session security, Jinja2 templating, and request handling.

- **MySQL 8.0 Reference Manual**

  `https://dev.mysql.com/doc/refman/8.0/en/`

  InnoDB storage engine, connection pooling, and row-level locking architecture.

- **W3C Media Capture Streams API**

  `https://www.w3.org/TR/mediacapture-streams/`

  HTML5 WebRTC standard for client-side webcam streaming and photo capture.

THANK YOU

FarmDirect – A Farmer-to-Consumer Agricultural Marketplace

QUESTIONS & VIVA VOCE DISCUSSION

We welcome questions, suggestions, and feedback on the project architecture, concurrency control models, and software engineering implementation.

NAME OF STUDENT : SANTHOSH RATHINAM M

ENROLMENT NUMBER : EA2532251010685

MASTER OF COMPUTER APPLICATIONS (MCA - III SEMESTER)

Directorate of Online Education, SRM Institute of Science and Technology